

## 6

## Building the Summarizer Agent for Context Reduction

This chapter marks an important transition, tackling one of the core business challenges in enterprise AI: managing the financial and technical costs of large contexts. We will confront this challenge through a new capability, **context reduction**, by introducing a specialist agent called the **Summarizer**. Its purpose is to intelligently reduce content volumes, acting as a gatekeeper against practical constraints such as token limits and API costs.

Before writing a single line of code, we will revisit the system's architectural blueprint and conceptually design a new capability: **proactive context management**. We will visualize how the Summarizer integrates into the existing framework. With a clear plan established, we will move to implementation, translating the design into a self-contained agent and integrating it into the engine's discoverable toolkit. By the end of this chapter, you will not only have built a new type of context-reduction agent, but also deepened your understanding of what it means to think like a context engineer. You will learn how to analyze a business constraint, architect an efficient solution, and implement it within a **multi-agent system (MAS)**. These are the skills that elevate an AI practitioner from operator to architect, someone capable of building truly intelligent, enterprise-grade systems.

This chapter covers the following topics:

- Architecting the glass-box system
- Implementing context reduction with the Summarizer agent
- Translating technical efficiency into business value

### Architecting the glass-box system

As with our previous chapters, we begin with an architectural walk-through to visually and conceptually ground ourselves before moving into implementation. To adapt to the new skills, we must learn to increasingly think about AI systems before writing code. The most advanced AI copilots can now boost code generation, but they cannot design a system without a human thinking about the design. We humans remain the architects, and the AI copilots are workers, as shown in this figure:

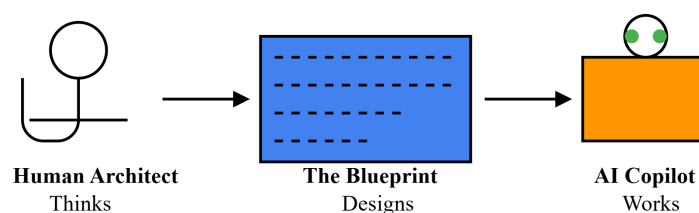


Figure 6.1: From human architects to AI workers

With this architect's mindset, we must address a critical choice every AI engineer faces. Many rush to use AI as a simple black box that provides an input and then hope for a quality output. We will take a more professional and powerful approach by enforcing a **glass-box system**, deliberately designed for transparency and control. Unlike a *black box*, where the internal logic is hidden, our context engine is built to be understood. We began implementing glass-box functionality in [Chapter 4](#), making every action traceable through a detailed `ExecutionTrace`. That way, every component is modular, and every decision the AI makes can be examined. We will do the same when building content-reduction features. You might think: "Why bother?" The answer is simple. Everybody is rushing to chatbots to boost productivity. However, the quality of the outputs of an AI copilot is strictly linked to the quality of our inputs; hence, a necessity to develop a glass-box engine for full control. The context engine, as hardened in [Chapter 5](#), is a sophisticated system with a sequential flow of operations. Understanding this glass-box architecture is the key to appreciating how seamlessly our new Summarizer agent will integrate into its existing logic.

In this section, we will first walk through the operational flow of the engine, from the initial user goal to the final generated output. Then, we will pinpoint exactly where and why our new components will be introduced. This two-layered approach will provide a complete picture of the engine's evolution from a powerful orchestrator into an efficiency-conscious system. Let's design the architectural blueprint of the content reduction process.

## Step-by-step architectural walkthrough

Our guide for this walkthrough is *Figure 6.2*. Each node in the diagram is color-coded to match the corresponding sections in our code. The new components introduced in this chapter are clearly marked with a **[NEW]** label.

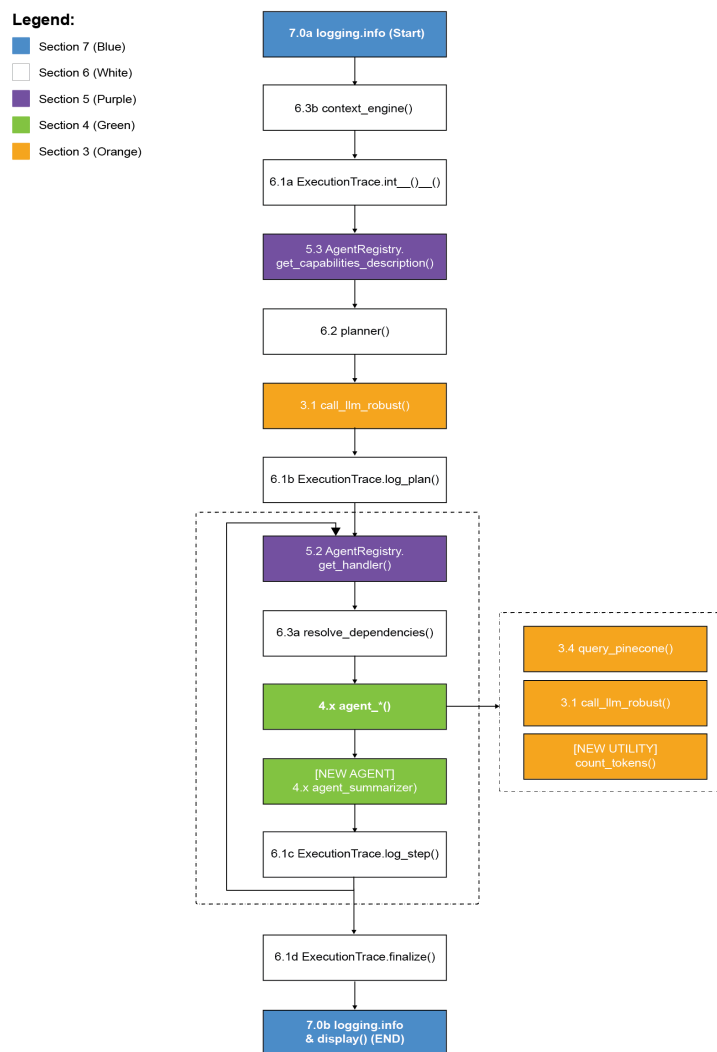


Figure 6.2: The content reduction architecture

The legend of *Figure 6.2* contains a color code for each component of the context engine:

- **Blue (section 7):** The final execution script that the user runs
- **White (section 6):** The engine's core, containing the main orchestrator, planner, and tracer
- **Purple (section 5):** The agent registry, which acts as the system's toolkit
- **Green (section 4):** The specialist agents who perform the actual work
- **Orange (section 3):** The helper functions that provide common utilities

Just looking at the flowchart can be daunting! The first thought that comes to mind is, “When does this end?” We’ve already gone through five chapters, and yes, things are getting more complex. But that’s exactly the point. An AI architect, a context engineer, is a *complexity thinker*. Many assume it isn’t worth the effort to build a system from scratch, rushing instead to ready-made platforms that promise effortless implementation with polished examples. But the reality is far less glamorous. Every advanced AI project, without exception, hits hard walls. Sometimes data is missing and progress halts; sometimes key functions don’t behave as expected; and sometimes, inevitably, bugs appear at the worst possible moment.

So, what happens then? That’s when the real AI engineer steps forward, someone who understands the architecture at its core, who knows how a MAS or MCP-based context engine works at the lowest level. When that moment comes, everyone turns to the expert and not to the confused operator still staring at the dashboard, wondering what went wrong.

Now that we’ve seen the full blueprint, let’s walk through it in motion. *Figure 6.2* comes alive when we follow the path of a single task through the following four operational phases, from user input to final output. Understanding this flow will make it easier to see exactly where our new Summarizer agent fits in and how it enhances the engine’s capabilities:

1. **Initiation:** Everything begins when the user runs the final code block. The `context_engine()` function serves as the entry point, the heart of the orchestrator that takes control of the entire process. It’s where intent is translated into motion, as we have established in previous chapters.
2. **Planning:** Once inside `context_engine()`, the first priority is to design a plan of action. An `ExecutionTrace` object is created to serve as the engine’s *flight recorder*, logging every step for transparency and debugging. The `planner()` then needs context—it queries `AgentRegistry` via `get_capabilities_description()` to understand what specialist agents are available. Armed with this list and the user’s goal, the planner invokes the LLM using `call_llm_robust()`, which returns a structured, multi-step JSON plan that defines the overall strategy.
3. **Execution loop:** With a clear plan in place, the engine shifts into execution mode. It iterates through each step, retrieving the corresponding agent function from the registry using `get_handler()`. Dependencies are resolved through `resolve_dependencies()`, which performs context chaining by replacing placeholders (such as `$$ STEP_1_OUTPUT $$`) with the actual outputs from previous steps. The ready-to-run agent is then invoked to perform its task, often relying on helper functions such as `query_pinecone()` or `call_llm_robust()` for data retrieval or reasoning. Once a step completes, its results are logged into the trace, and the engine moves seamlessly to the next.
4. **Finalization:** After all steps have been executed, the engine finalizes the process by calling `finalize()` on the trace object. This records the final status and execution time before returning a clean, structured output to the user.

Our context engine is now operating like a true team of intelligent collaborators, autonomous and agent-driven. Each component carries a clear responsibility, just as members of a well-coordinated team do. Let’s now explore these responsibilities more closely to see how the system keeps its balance as it grows in complexity.

## Separation of responsibilities

The following sequential flow reveals a clear separation of responsibilities. Each module is represented here using the same color labels shown

in *Figure 6.2* to help you visually connect the concepts explained in the architecture diagram. It's within this modular framework that we now introduce our new components and upgrades:

- **Engine core (white)** is the brain. It manages the entire process but doesn't do the work itself. Its logic is generic and robust, and therefore requires no changes in this chapter.
- **Specialist agents (green)** are the workers. Each is responsible for a specific task within the system. We will introduce a new agent (highlighted as **[NEW AGENT]** in *Figure 6.2*), `agent_summarizer`. Its role is to serve as an intelligent gatekeeper, taking large volumes of text and reducing them to their essential points based on a specific objective. This agent directly addresses the business need for managing API costs and operating within fixed token limits.
- **Agent registry (purple)** is the toolkit. It provides the Planner with a descriptive manual of available tools. The registry's manual (`get_capabilities_description`) will be updated to include our new Summarizer agent, making the Planner instantly aware of this new capability.
- **Helper functions (orange)** are the shared utilities, the quiet enablers that keep everything running smoothly. We will formally acknowledge the new utility (highlighted as **[NEW UTILITY]** in *Figure 6.2*, the `count_tokens` function. This function acts as the engine's fuel gauge, allowing any component to measure the token cost of a piece of text *before* sending it to an LLM. It is the foundational tool that enables a proactive, cost-conscious design.

Now that we understand the base architecture and where our new components will fit, let's summarize the strategic impact of these changes in a way that is clear for any stakeholder, technical or not. As context engineers, we must go beyond building functional systems and must learn to *communicate* their evolution in a language that makes sense to decision-makers. Each upgrade, no matter how technical, should be explainable in conceptual terms that bridge engineering logic and business value:

- **Phases 1 and 2—Proactive context management:** Our primary upgrade is not just a new agent, but a new strategic capability: proactive context management. Instead of simply processing whatever information it is given, the engine can now analyze, measure, and condense it. The `count_tokens` utility provides the measurement, and the Summarizer agent provides the action. This transforms the engine from being purely reactive to being strategically efficient, allowing it to tackle larger problems more economically.
- **Phase 3—Integration and scalability:** The most powerful aspect of our architecture is that this significant new capability can be added without redesigning the core engine. By simply adding the Summarizer agent and updating the agent registry's description, the Planner automatically learns how to use this new tool. This proves that the system is highly scalable; new agents and capabilities can be added in the future with minimal disruption, much like adding a new tool to a well-organized workshop.
- **Phases 4 and 5—Validation:** The final part of our process is to prove that these upgrades work and, just as importantly, that they haven't broken any existing functionality. We will conduct rigorous tests, including a backward compatibility check, to validate that the engine remains stable and that the Planner can still formulate simpler plans correctly. This focus on validation is essential for building trust in a system intended for enterprise use.

We have covered a lot of architectural thinking ground now, but we need to be able to explain *why* this focus on validation is essential for building trust in a system intended for enterprise use.

## Why the glass box matters

The clear separation of responsibilities, as depicted in our flowchart, is not merely an elegant design choice, nor is the conceptual overview of

the upgrades some content to overlook before rushing to the code. It is a strategic imperative for any serious AI engineering project. Why rush to the code when AI copilots are progressively occupying that role? The most recent and powerful LLMs can produce reasonably good production-level code. So, take your time before going to the code and learn why the glass-box architecture we are building will take you to the next level of AI expertise.

First and foremost, this design actively prevents technical debt. In a tightly-coupled or monolithic system, adding a new capability often requires complex and risky changes to the core logic. Over time, the system becomes a brittle *house of cards*, where a single modification can cause cascading failures. Our architecture, by contrast, treats agents as plug-and-play modules. Adding the Summarizer does not require us to alter the fundamental workings of the Executor or Planner. This is analogous to upgrading a car's GPS; you can replace the unit without having to re-engineer the engine. This principle ensures that our context engine remains agile and adaptable for years to come.

Furthermore, this modularity is an enabler for effective teamwork and scalability. Imagine a scenario where a business needs to deploy agents for its legal, financial, and marketing departments. With our architecture, three separate developers or teams can work on `agent_legal_parser`, `agent_financial_analyzer`, and `agent_marketing_writer` in parallel. Their only requirements are to adhere to the MCP for communication and to register their finished agents in the central Agent Registry. They do not need to understand the intricate details of the engine's core orchestration, dramatically lowering the barrier to contribution and accelerating the development of new business capabilities. You are guiding your teams into a new era!

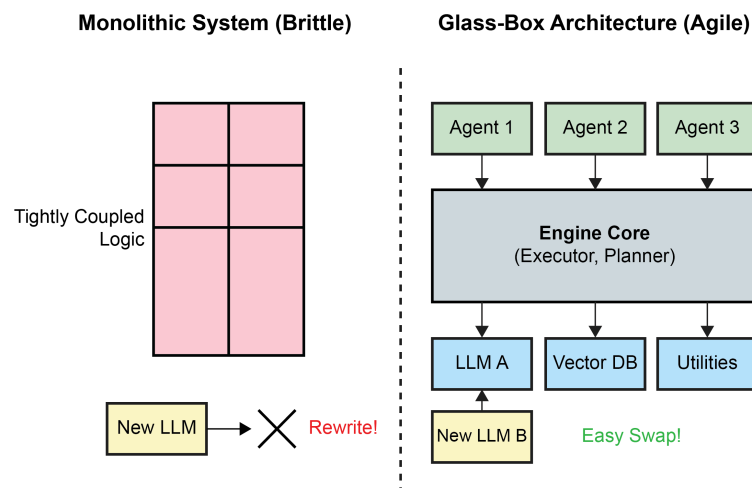


Figure 6.3: From brittle to agile AI systems

Figure 6.3 illustrates the major shift for you as an emerging context engineer architect from the former prompt and classical AI system design. This visual metaphor represents more than just a choice between two software design patterns; it encapsulates a fundamental shift with critical implications for systems and humans alike.

For the AI system, this is the evolution from a static, single-purpose tool into a resilient, living ecosystem. Its value is no longer measured solely by

its immediate output, but by its capacity to adapt, grow, and integrate new capabilities over time without collapsing.

For the human engineer, this shift is even more significant. It elevates the role from a builder of isolated applications to an architect of intelligent systems. As AI copilots increasingly handle the task of generating code, the most critical human skill becomes strategic foresight: the ability to design frameworks that are not just powerful today but are agile enough to embrace the inevitable advancements of tomorrow. This agile approach, where we architect for change, is the very essence of modern context engineering.

With this clear architectural and conceptual map in hand, we are ready to begin the hands-on implementation.

## Implementing context reduction with the Summarizer agent

With a firm grasp of our engine's upgraded architecture, we have established the solid foundation upon which we will build this new capability. We will now transition from architectural theory to practical implementation. This implementation journey will be a methodical process, requiring precise additions to our core library files to seamlessly integrate the new agent.

Our primary workspace for orchestrating and testing these enhancements is the dedicated notebook called

`Context_Engine_Content_Reduction.ipynb`, which is an upgraded version of `Chapter05 / Context_Engine_Pre-Production.ipynb`. The installation libraries remain the same. To maintain version clarity and keep the project structure clean as the engine evolves, each chapter's code now resides in its own subdirectory. For this chapter, all shared library files are sourced from `commons/ ch6 /`, a clear and organized space to manage upgrades as we continue to scale the context engine.

The upgrades will proceed through five methodical phases, building upon the robust architecture we established in [Chapter 5](#):

- **Phase 1—Foundation for cost management:** Begin by verifying the `count_tokens` utility, which enables the system to proactively measure and manage context size.
- **Phase 2—Building the summarizer agent:** Construct the core logic for the new agent in `agents.py`, designing it as a self-contained, dependency-aware module.
- **Phase 3—Integrating the agent into the engine toolkit:** Update `registry.py` so the Planner LLM becomes aware of the new capability and can incorporate summarization dynamically within its workflows.
- **Phase 4—Reinforcing the Writer agent:** Upgrade `agent_writer` to accept outputs from both Researcher (`{"facts": ...}`) as well as the new Summarizer (`{"summary": ...}`).
- **Phase 5—Demonstrating the new capability:** Execute a new, complex goal to showcase how the Summarizer agent intelligently optimizes performance and reduces cost in a real-world scenario.

Our first step will be to examine cost management.

### Foundation for cost management

Our work in this chapter centers on managing context size, which first requires the ability to measure it accurately. We will verify the `count_tokens` utility that was added to our `helpers.py` library during the initial hardening process.

As mentioned earlier, this utility serves as the engine's fuel gauge, providing a precise measurement of how many tokens a given piece of text will consume before it is sent to the LLM. It is a simple but critical function that lays the groundwork for all subsequent cost-management strategies. We will now examine its implementation:

```
# FILE: commons/helpers.py (existing code)
# This function is our primary tool for proactive token management.

def count_tokens(text, model="gpt-4"):
    """Counts the number of tokens in a text string for a given model."""
    try:
        encoding = tiktoken.encoding_for_model(model)
    except KeyError:
        # Fallback for models that might not be in the tiktoken registry
        encoding = tiktoken.get_encoding("cl100k_base")
    return len(encoding.encode(text))
```

The `count_tokens` function accepts a string of text and an optional model name. It uses the `tiktoken` library, the official tokenizer from OpenAI, to encode the text into a list of token integers and then returns the length of that list. By wrapping the encoding process in a `try...except` block, it gracefully handles cases where a specified model may not be in the library's registry, falling back to a general-purpose base encoding to ensure that the function always returns a value.

With our foundational measurement tool verified, we can now build the agent that will use this principle to manage context. We will now proceed to construct the Summarizer agent.

## Building the Summarizer agent

The Summarizer agent is our solution to the problem of excessive context. Its purpose is to intelligently reduce large bodies of text into concise summaries that are tailored to a specific objective, ensuring that only the most relevant information is passed on to subsequent agents in the workflow.

We will add this new agent to our `agents.py` file, following the same production-ready patterns we established for our other agents. It will be a self-contained function that receives all its dependencies as arguments and communicates its results using MCP. The agent's intelligence lies in its use of `summary_objective`, which directs the LLM to create a summary that is not just shorter, but more useful.

The function begins by defining its signature, `agent_summarizer`, which accepts the standard `mcp_message` and its required dependencies: the `client` object for making API calls and the `generation_model` configuration. Inside the `try` block, it unpacks its two required inputs from the MCP message content: `text_to_summarize`, which contains the large block of text, and `summary_objective`, which provides the specific goal for the reduction:

```
# FILE: commons/ch6/agents.py
# This new function is added to our existing agent library.
# It follows the established dependency injection and structured logging patterns.

def agent_summarizer(mcp_message, client, generation_model):
    """
    Reduces a large text to a concise summary based on an objective.
    Acts as a gatekeeper to manage token counts and costs.
    """
    logging.info("[Summarizer] Activated. Reducing context...")
    try:
        # Unpack the inputs from the MCP message
        text_to_summarize = mcp_message['content'].get('text_to_summarize')
        summary_objective = mcp_message['content'].get('summary_objective')
```

The agent includes a validation step to ensure that both required inputs are present, raising a `ValueError` if they are not. It then constructs a `system_prompt` that instructs the LLM to act as an expert summarizer. The `user_prompt` is assembled using `f`-strings, clearly delineating `OBJECTIVE` and `TEXT TO SUMMARIZE`. This structured prompting is critical for guiding the LLM to produce a high-quality, relevant summary:

```
# The agent validates that it has received the necessary inputs before proceeding.

if not text_to_summarize or not summary_objective:
    raise ValueError("Summarizer requires 'text_to_summarize' and 'summary_objective'")

# Define the prompts for the LLM
system_prompt = """You are an expert summarization AI. Your task is to reduce the
user_prompt = f"""--- OBJECTIVE ---\n{summary_objective}\n\n--- TEXT TO SUMMARIZE
```

Finally, the agent calls our hardened `call_llm_robust` helper function, passing the prompts and the injected dependencies. The resulting summary is then wrapped in a standard MCP message using the `create_mcp_message` helper. The output is a dictionary with the key `"summary"`, ensuring a predictable data structure for downstream agents. The entire process is enclosed in a `try...except` block for robust error handling:

```
# The agent calls the robust LLM helper function and returns the result.

# Call the hardened LLM helper to perform the summarization
summary = call_llm_robust(
    system_prompt,
    user_prompt,
    client=client,
    generation_model=generation_model
)

# Return the summary in the standard MCP format
return create_mcp_message("Summarizer", {"summary": summary})
except Exception as e:
    logging.error(f"[Summarizer] An error occurred: {e}")
    raise e
```

Let's stop a minute and think back. We integrated a new agent smoothly by validating inputs, guiding the LLM with a clear objective, returning a predictable MCP message, and wrapping the call in `try...except`. That is what good context engineering looks like.



## Micro-context engineering

While the Python code for the Summarizer agent is straightforward, its true power lies not in the function itself, but in the quality of the context we provide to `summary_objective`. Crafting a precise objective is a perfect example of **micro-context engineering**. It is the skill of turning a generic, multi-purpose tool into a specialized, high-precision instrument for a specific task. An LLM, like any powerful tool, performs best when given clear, unambiguous instructions.

A good context engineer supplies solid, purposeful inputs; a poor one never obtains the expected results. So, keep your eyes open. Focus on upskilling your context design abilities. *Make sure an instruction is not an old-fashioned prompt!* Consider the vast difference in the quality of output that results from a poorly-defined objective versus a well-architected one.

A **poor objective** is vague and open-ended:

```
"Summarize this text."
```

This forces the LLM to guess what the user deems important. The result is often a generic, bland paragraph that may completely miss the critical information the user actually needed, wasting both the API call and the user's time.

A **strong objective**, by contrast, is a miniature semantic blueprint. It is specific, provides constraints, and clearly defines the desired output:

```
"Extract the names of all involved parties, the key financial figures discussed, and the f
```

This objective transforms the generic `Summarizer` into a precision entity-extraction tool for this one specific task. It leaves no room for ambiguity and guarantees a useful, structured output. Mastering the art of crafting these objectives is a core competency for a context engineer, as it allows them to dynamically repurpose a single, reusable agent for a multitude of specific, high-value business functions.

Again, we are witnessing the paradigm shift from development to design. We will not be replaced by AI. If we are good context engineers, we will take our AI workers to the next level as we upskill. The poor AI engineers will produce low-value output, as illustrated in *Figure 6.4*.

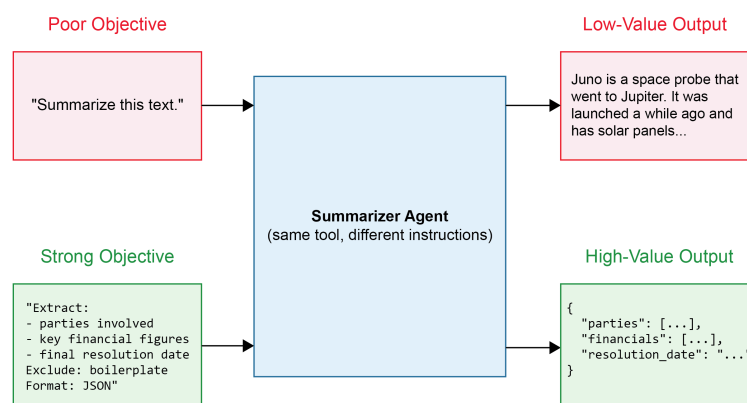


Figure 6.4: The importance of context engineering skills

The Summarizer agent is now built, but it exists in isolation. We must now update the engine's central toolkit to make this new capability discoverable.

## Integrating the new agent into the engine's toolkit

With the Summarizer agent built and tested in isolation, our next step is to make it discoverable by the system. An agent that the Planner does not know about will never be used. `AgentRegistry` serves as the discoverable toolkit for the entire context engine. To complete our upgrade, we must update it to include the new Summarizer agent.

This process involves two precise modifications to the `registry.py` file:

1. Add the `agent_summarizer` function to the registry's internal dictionary.
2. Update the `get_capabilities_description()` method, the plain-text *manual* that the Planner reads to understand what tools exist and how to use them.

This second step is crucial because it enables dynamic planning: the Planner learns new capabilities simply by reading this description rather than by being hardcoded.

In the `__init__` method of the `AgentRegistry` class, we add a new key-value pair to the `self.registry` dictionary. The key is the agent's name, `"Summarizer"`, which will be used in the execution plans, and the value is the callable function we just created, `agents.agent_summarizer`:

```
# FILE: commons/ch6/registry.py
# The entire file is updated to integrate the new agent.

# === Imports ===
import logging
import agents
from helpers import create_mcp_message

# === 5. The Agent Registry (Final Hardened Version) ===
class AgentRegistry:
    def __init__(self):
        self.registry = {
            "Librarian": agents.agent_context_librarian,
            "Researcher": agents.agent_researcher,
            "Writer": agents.agent_writer,
            # --- NEW: Add the Summarizer Agent ---
            "Summarizer": agents.agent_summarizer,
        }
```

Next, we add a new `elif` block to the `get_handler` method. This ensures that when `Executor` requests `"Summarizer"`, the registry correctly prepares it with the `client` and `generation_model` dependencies it requires to function. This adheres to our strict dependency injection pattern:

```
# The get_handler method is updated to prepare the new agent with its dependencies.

def get_handler(
    self, agent_name, client, index, generation_model,
    embedding_model, namespace_context, namespace_knowledge
):
    handler_func = self.registry.get(agent_name)
```

```

if not handler_func:
    logging.error(f"Agent '{agent_name}' not found in registry.")
    raise ValueError(f"Agent '{agent_name}' not found in registry.")

# --- UPDATED: Add a condition for the Summarizer ---
if agent_name == "Librarian":
    return lambda mcp_message: handler_func(
        mcp_message, client=client, index=index,
        embedding_model=embedding_model,
        namespace_context=namespace_context
    )
elif agent_name == "Researcher":
    return lambda mcp_message: handler_func(
        mcp_message, client=client, index=index,
        generation_model=generation_model,
        embedding_model=embedding_model,
        namespace_knowledge=namespace_knowledge
    )
elif agent_name == "Writer":
    return lambda mcp_message: handler_func(
        mcp_message, client=client,
        generation_model=generation_model
    )
elif agent_name == "Summarizer":
    return lambda mcp_message: handler_func(
        mcp_message, client=client,
        generation_model=generation_model
    )
else:
    return handler_func

```

The final and most critical update is to the docstring returned by `get_capabilities_description`. We add a new entry for the Summarizer. We clearly define its role, its required inputs ( " text\_to\_summarize " and " summary\_objective " ), and the structure of its output. This detailed description is precisely what allows the Planner to understand when and how to strategically deploy the Summarizer agent in its plans:

```

# Finally, we update the capabilities description that the Planner LLM reads.

def get_capabilities_description(self):
    """Returns a structured description of the agents for the Planner LLM."""
    # --- UPDATED: Add the Summarizer's capabilities ---
    return """
Available Agents and their required inputs.
CRITICAL: You MUST use the exact input key names provided for each agent.

1. AGENT: Librarian
ROLE: Retrieves Semantic Blueprints (style/structure instructions).
INPUTS:
- "intent_query": (String) A descriptive phrase of the desired style.
OUTPUT: The blueprint structure (JSON string).

2. AGENT: Researcher
ROLE: Retrieves and synthesizes factual information on a topic.
INPUTS:
- "topic_query": (String) The subject matter to research.
OUTPUT: Synthesized facts (String).

3. AGENT: Summarizer
ROLE: Reduces large text to a concise summary based on a specific objective. Ideal for
INPUTS:
- "text_to_summarize": (String/Reference) The long text to be summarized.

```

```

- "summary_objective": (String) A clear goal for the summary (e.g., "Extract key tech
OUTPUT: A dictionary containing the summary: {"summary": "..."}).

4. AGENT: Writer
ROLE: Generates or rewrites content by applying a Blueprint to source material.
INPUTS:
- "blueprint": (String/Reference) The style instructions (usually from Librarian).
- "facts": (String/Reference) Factual information (usually from Researcher or Summari
- "previous_content": (String/Reference) Existing text for rewriting.
OUTPUT: The final generated text (String).
\\\\"

```

Figure 6.5 shows the dynamic, four-step discovery process that enables the Planner LLM to learn about and strategically use new agents:

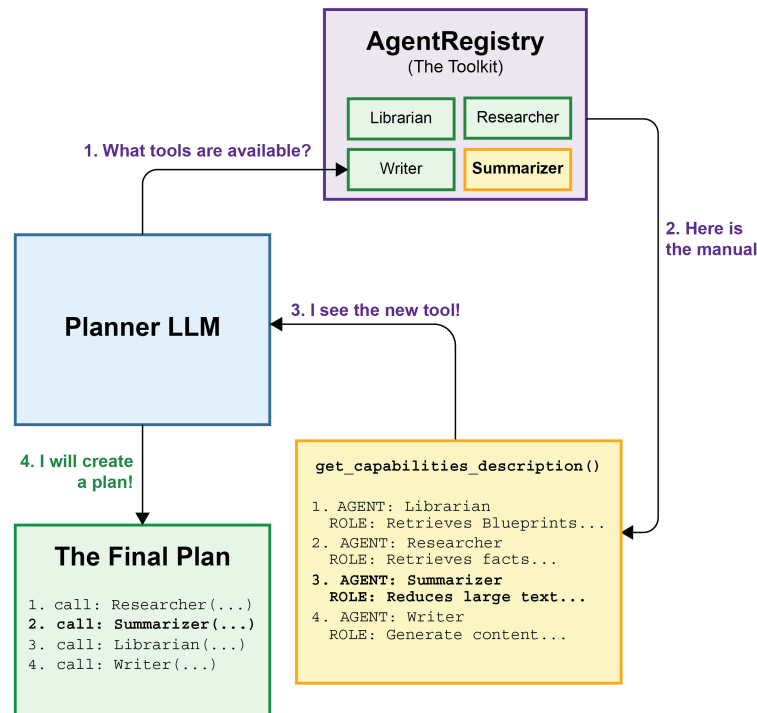


Figure 6.5: The dynamic discovery loop between the Planner and AgentRegistry

The diagram illustrates how the `get_capabilities_description()` method forms the bridge between the Planner and AgentRegistry. The process begins when the Planner (the brain) asks the Agent Registry (the toolkit) what tools are currently available (1). The registry responds, not with code, but with a clear, plain-text manual describing each agent's role and inputs (2). By reading this manual, the Planner discovers all the available agents, including the newly added Summarizer (3). Equipped with this knowledge, the Planner can then construct a well-informed, multi-step plan that uses the right tools in the right order to achieve the user's goal (4). This decoupled discovery loop, where the Planner learns dynamically from the registry, is what makes the system scalable, adaptable, and truly intelligent.

Our new agent is now fully integrated into the context engine's architecture. Let's now reinforce the Writer agent for flexibility.

## Reinforcing the Writer agent for greater flexibility

Our new Summarizer agent is now fully integrated into the engine's tool-kit. However, a key principle of robust system design is ensuring that new components can collaborate effectively with existing ones. The introduction of the Summarizer creates a new potential data source for our Writer agent, which was previously only designed to work with the Researcher. To make our engine truly dynamic, we must upgrade the Writer to be more adaptable, enabling it to seamlessly accept input from either of its potential collaborators.

This upgrade embodies a core concept of advanced system design: building components that are not only powerful in their own right but also flexible in their interactions. We will now reinforce our Writer agent to handle multiple data formats, making our entire MAS more resilient and intelligent. Let's look into the data contract challenge.

The need for this upgrade was revealed during initial testing. The Planner, in its emergent wisdom, correctly chained the output of the new Summarizer to the Writer. However, this created a **data contract violation**. The two information-providing agents had slightly different ways of formatting their output data:

- Researcher o utput: {'facts': '...'}
- Summarizer o utput: {'summary': '...'}

The original Writer agent was only built to understand the first schema. When it received a dictionary with a 'summary' key instead of a 'facts' key, it failed. To solve this, we will make the Writer *bilingual*, capable of understanding both data formats.

We will now modify the data unpacking logic at the beginning of the `agent_writer` function in our `commons/ ch6 / agents.py` file. The upgrade is simple but powerful, teaching the Writer to check for multiple possible input keys.

Here is the complete code for the newly reinforced `agent_writer`:

```
# FILE: commons/ch6/agents.py (UPGRADED agent_writer)

def agent_writer(mcp_message, client, generation_model):
    """Combines research with a blueprint to generate the final output."""
    logging.info("[Writer] Activated. Applying blueprint to source material...")
    try:
        blueprint_data = mcp_message['content'].get('blueprint')
        facts_data = mcp_message['content'].get('facts')
        previous_content = mcp_message['content'].get('previous_content')
```

The function signature and initial unpacking of the main MCP content parts (`blueprint`, `facts`, and `previous_content`) remain the same as before:

```
# UPGRADE: Robust logic for handling multiple data contracts
blueprint_json_string = blueprint_data.get('blueprint_json')
if isinstance(blueprint_data, dict) else blueprint_data

facts = None
if isinstance(facts_data, dict):
    # First, try to get 'facts' (from Researcher)
    facts = facts_data.get('facts')
```

```

        # If that fails, try to get 'summary' (from Summarizer)
        if facts is None:
            facts = facts_data.get('summary')
        elif isinstance(facts_data, str):
            facts = facts_data

    if not blueprint_json_string or (not facts and not previous_content):
        raise ValueError("Writer requires a blueprint and either 'facts' or 'previous_

```

This block contains the critical upgrade. After extracting `blueprint_json_string`, we introduce new logic to find the factual content. The code first checks whether the `facts_data` it received is a dictionary. If so, it first attempts to get a value using the `'facts'` key. If that returns `None`, it makes a second attempt using the `'summary'` key. This simple conditional check makes the agent instantly compatible with the output of both the Researcher and the Summarizer:

```

        # Determine the source material and label for the prompt
    if facts:
        source_material = facts
        source_label = "SOURCE FACTS"
    else:
        source_material = previous_content
        source_label = "PREVIOUS CONTENT (For Rewriting)"

    # Construct the prompts and call the LLM...
    system_prompt = f"""You are an expert content generation AI...""" # (prompt remain
    user_prompt = f"""--- SOURCE MATERIAL ({source_label}) ---\n{source_material}\n---

    final_output = call_ll_robust(
        system_prompt, user_prompt, client=client,
        generation_model=generation_model
    )
    return create_mcp_message("Writer", final_output)

except Exception as e:
    logging.error(f"[Writer] An error occurred: {e}")
    raise e

```

The remainder of the function proceeds as before. It determines which data to use as `source_material`, constructs the detailed system and user prompts, and calls the LLM to generate the final content.

With our Writer agent now reinforced to handle multiple data sources, we are ready to run our first end-to-end test of the new context reduction workflow.

## Exploring the Summarizer in action

The final phase of our implementation is to demonstrate the new context reduction workflow. We will design a goal that makes the use of the Summarizer not just beneficial, but necessary for the successful completion of the task.

---

LLMs are stochastic models. As such, the output may vary from one run to the next.

---

The goal will be twofold: it will first explicitly ask the engine to summarize a large piece of provided text for a specific objective, and then it will ask the engine to use that summary as the factual basis for a creative writing task. This complex structure forces the Planner to create a multi-step plan that leverages context chaining, using the output of the Summarizer as the input for the Writer. This test will provide definitive proof that our new agent is fully integrated and that the engine can deploy it dynamically.

First, we define a string variable, `large_text_from_researcher`, which contains a dense paragraph of technical information about the Juno space probe. We then construct our goal using an `f`-string. The goal explicitly instructs the engine to perform two sequential tasks. First, summarize the provided text for a specific objective, and then use that summary for a creative writing task:

```
# FILE: Context_Engine_Content_Reduction.ipynb
# This is the final demonstration from the main notebook's control deck.

# 1. Define a large piece of text that would be expensive or too long
# to use as direct context for the Writer agent.
large_text_from_researcher = """
Juno is a NASA space probe orbiting the planet Jupiter. It was launched from Cape Canavara
"""

# 2. Define a goal that requires both using the large text AND a creative step.
goal = f"""First, summarize the following text about the Juno probe to extract only the ke

--- TEXT TO USE ---
{large_text_from_researcher}
"""
```

We use the same `config` dictionary as in our previous run and pass all the necessary components to the `execute_and_display` function:

```
# The configuration remains the same, and the execution call is made.

# 3. Use the same configuration dictionary
config = {
    "index_name": 'genai-mas-mcp-ch3',
    "generation_model": "gpt-5",
    "embedding_model": "text-embedding-3-small",
    "namespace_context": 'ContextLibrary',
    "namespace_knowledge": 'KnowledgeStore'
}

# 4. Call the execution function
execute_and_display(goal, config, client, pc)
```

Upon running this cell, we can analyze `TECHNICAL TRACE` in the output to deconstruct the engine's thought process. The trace reveals a multi-step plan, beginning with the Summarizer agent, which successfully reduces the large text into a concise, bulleted list of key facts. Crucially, the trace shows that the output of this first step is then passed, via context chaining, as the `facts` input to the Writer agent in a subsequent step. The final output is a creative story that is factually grounded in the *summary*, not the original large text, proving that our context reduction workflow was

executed successfully. This confirms that the Summarizer is a fully functional and strategically valuable addition to our MAS.

Let's go through the final output to make sure our implementation has worked as expected.

## Deconstructing the engine's thought process: the proof is in the trace

The final generated text is compelling, but the true measure of our success lies within `TECHNICAL TRACE`. Being the engine's flight recorder, the trace provides a transparent, step-by-step account of its reasoning and actions. By deconstructing this trace, we can see precisely how the Planner formulated a novel strategy to meet our complex goal and how the Summarizer agent played its critical role in enabling the final output.

We will now walk through the execution trace from our test run, analyzing each decision and action to confirm that our new, efficiency-focused architecture is performing exactly as designed, beginning with the Summarizer agent.

### Step 1: Summarizer agent takes the lead

The very first action the Executor takes is to call the Summarizer agent. This is the most important validation of our upgrade. The Planner, when presented with the user's goal, analyzed the request and recognized two key components: a large, embedded block of text and an explicit instruction to "summarize the following text." Based on the updated capabilities description we provided in `registry.py`, the Planner correctly concluded that the Summarizer agent was the ideal tool for the first step. It did not default to the Researcher because the factual context was already provided within the goal itself. This is a clear demonstration of dynamic planning.

The Planner intelligently extracted the user's instructions and formulated a highly specific objective for the summary:

```
'summary_objective': 'Extract only the key facts about '
                    'Juno's scientific mission and its '
                    'instruments/power system...'
```

The Summarizer agent then executed its task, calling the LLM with this context and objective. Its output is a concise, bullet-pointed list containing only the most relevant facts about Juno's mission:

```
'output': { 'summary': '- Operates in a polar orbit around Jupiter...'
            '- Measures Jupiter's composition, gravitational field...'
            '- Investigates formation by probing the interior...'
            '- Determines the amount of water in the deep atmosphere...'
            '- Maps mass distribution and characterizes deep winds...'
            '- Powered and stabilized by three very large solar-array wings...'
          }
```

Here, we see the direct economic and technical value of the agent. Using our `count_tokens` utility separately in a function, we can quantify the reduction:

- **Original text** ( `text_to_summarize` ): 253 tokens
- **Final summary** ( `output` ): 110 tokens

The Summarizer agent achieved a 56.5% reduction in token count. It successfully filtered the high-volume, low-relevance details (such as launch dates and comparisons to the Galileo orbiter) and passed on a low-volume, high-relevance summary. This action directly translates to lower API costs and ensures that the most critical information is preserved for the subsequent, more expensive generation step.

There are many ways and places to count tokens in the context engine throughout the process that should be decided in architecture workshops during the development phase. For example, in this case, you could include the token count as a post execution function:

```
# === Post-Execution Analysis: Quantifying Context Reduction ===

# Make sure to import the count_tokens utility
from helpers import count_tokens

# 1. Get the original text that was sent to the Summarizer
# (This is the same variable we defined in the control deck)
original_text = large_text_from_researcher

# 2. Get the summarized text from the trace object
# The trace object 'trace_1' was returned by the execute_and_display function.
# We look inside the first step (index 0) of the execution trace.
summarized_text = trace_1.steps[0]['output']['summary']

# 3. Use the 'count_tokens' utility to measure both
original_tokens = count_tokens(original_text)
summarized_tokens = count_tokens(summarized_text)
reduction_percentage = (1 - (summarized_tokens / original_tokens)) * 100

# 4. Print the results
print("--- Context Reduction Analysis ---")
print(f"Original Text Tokens: {original_tokens}")
print(f"Summarized Text Tokens: {summarized_tokens}")
print(f"Token Reduction: {reduction_percentage:.1f}%")
```

This post execution function is one of many functions you can create depending on your project.

Now, let's move to *Step 2* and see how the Librarian agent continues the process.

### Step 2: Librarian secures the blueprint

With the factual basis for the story now established and condensed, the engine proceeds to the next part of the user's goal: the creative writing task. The Planner determined that to fulfill the request for a *short, suspenseful scene for a children's story*, it would need a stylistic guide.

The Executor calls the Librarian agent with an `intent_query` for a "short, suspenseful children's story scene." The Librarian searches the `ContextLibrary` namespace in our vector database and retrieves `blueprint_suspense_narrative`, which contains the stylistic rules for the Writer:

```
'output': { 'blueprint_json': '{
  "scene_goal": "Increase tension and create suspense.",
  "style_guide": "Use short, sharp sentences. Focus on sensory details...",
  ...
}'
}
```

This blueprint, with its `Use short, sharp sentences` and `Focus on sensory details` instructions becomes the stylistic instruction for the final step.

### Step 3: Writer synthesizes the final output

This final step is where the entire plan comes together and demonstrates the power of context chaining. The Executor activates the Writer agent, but before doing so, it resolves the dependencies specified in the plan.

By examining `resolved_context` for *Step 3* in the trace, we can see this process in action. The placeholders from the plan have been replaced with the actual outputs from the preceding steps:

```
'resolved_context': {
  'blueprint': { 'blueprint_json': '{"scene_goal": "Increase tension..."}'},
  'facts': { 'summary': '- Operates in a polar orbit...' },
  ...
}
```

This is the critical proof point. The Writer agent is not given the original 253-token article. It is given the lean, 110-token summary from the Summarizer. The agent now has everything it needs: a set of stylistic instructions (the blueprint) and a set of core facts (the summary). It calls the LLM one last time, instructing it to synthesize these two inputs into a final narrative.

The Summarizer has successfully acted as an intelligent gatekeeper, protecting the most expensive agent in the workflow from unnecessary context and cost. Let's go through the final output.

### Final output: Synthesis of style and summarized fact

The engine has successfully planned and executed a complex, multi-agent workflow. The final story is not a generic piece of creative writing; it is a direct and traceable synthesis of the components assembled by the preceding agents.

We can see the influence of both the Summarizer and the Librarian woven throughout the final text. The story is precisely grounded in the summarized facts provided by the Summarizer:

- The summary stated that Juno is "Powered and stabilized by three very large solar-array wings". The story begins with this fact, transformed into a first-person experience:

```
I drift toward the giant. Quiet. Focused. My three large solar array wings stretch wide. They hum. T
```

- The summary listed that Juno "Measures Jupiter's composition, gravitational field, magnetic field, and polar magnetosphere". The story internalizes this sci-

I listen with metal ears. I measure composition. I feel gravity tug and weigh it. I trace the magnet

Simultaneously, the narrative strictly adheres to the suspense blueprint retrieved by the Librarian. The style guide demanded "Use short, sharp sentences". The story's climax is a perfect execution of this rule, creating tension and pace:

Now. Flame. A steady roar. The craft shivers. Gravity clutches. I do not wobble. My wings

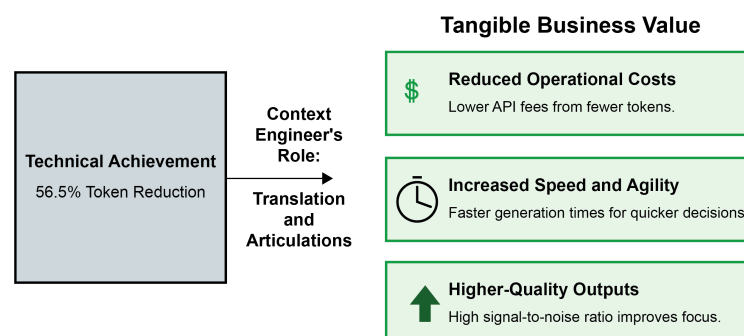
This detailed analysis of the trace and the final output provides definitive proof that our upgrades have been successful. We have created a new, valuable agent and seamlessly integrated it into our engine's dynamic planning capabilities. The context engine is now not only more powerful but also more efficient and cost-effective, ready to tackle even more demanding enterprise challenges.

This detailed analysis of the trace and the final output provides definitive proof that our upgrades have been successful. That's fine for an engineer, but we have yet to express that in business value.

## Translating technical efficiency into business value

A 56.5% reduction in token count is a commendable technical achievement, but a context engineer's responsibility extends to translating this efficiency into tangible business value. The ability to articulate the *so what* of a technical feature is what separates a developer from a strategic partner in the enterprise. This requires connecting abstract metrics such as token counts to concrete business outcomes such as cost, speed, and quality.

Consider a hypothetical but realistic business case: a financial services firm uses our context engine to process thousands of lengthy market analysis reports each day. The goal is to provide its traders with concise, actionable summaries. Let's quantify the impact of our Summarizer agent in this scenario. If the firm processes 10,000 reports daily, and our agent consistently reduces the token load on the final, most expensive generation step by over 50%, the financial implications are significant. This single agent could translate into direct operational savings of thousands, or even tens of thousands, of dollars per month in API costs alone. *Figure 6.6* shows that a new era AI engineer must be an architect, a context engineer, and also have the business skills to demonstrate the value of an AI project.



*Figure 6.6: New role of modern-era AI engineer is to prove business value of an AI project*

Furthermore, the value extends beyond cost savings, and we must be able to explain why. By providing the final Writer agent with a smaller, more focused context, we reduce the computational load, leading to faster generation times. This means traders get their critical information quicker, enabling more agile decision-making. The high signal-to-noise ratio of the summarized context can also lead to higher-quality outputs, as the final agent is less likely to be distracted by irrelevant details. This ability to frame a technical upgrade in terms of its impact on cost, speed, and quality is a crucial skill, allowing the context engineer to effectively champion their solutions and demonstrate their direct contribution to the business's bottom line.

Having successfully engineered an efficiency-focused solution and proven its value, it is time to summarize the critical skills and insights we have gained in this chapter.

## Summary

In this chapter, we successfully elevated the context engine to an economically efficient system by directly addressing the enterprise constraints of API costs and context limits. Our technical journey involved architecting and implementing a new specialist, the Summarizer agent, designed to act as an intelligent gatekeeper for large volumes of information. By integrating this agent into the engine's dynamic planning framework, we have equipped our system with a powerful new capability: proactive context management.

Our approach mirrored the professional workflow of a true context engineer, extending beyond mere coding. We began with a conceptual analysis of the architecture, methodically implemented the new agent, and integrated it into the discoverable Agent Registry. Crucially, we subjected our upgraded system to rigorous validation, performing backward compatibility checks to ensure new features did not compromise existing stability before proving the Summarizer's effectiveness on a complex, real-world task. Through this process, you have acquired skills that are fundamental to building enterprise-grade AI. You have learned how to architect for efficiency, how to safely extend a complex MAS, and how to analyze an execution trace to provide tangible proof of your system's value. We are ready to open our context engine to the outside world in the next chapter!

## Questions

1. Is the Summarizer agent's primary goal to add new facts from an external source? (Yes or no)
2. Does the `count_tokens` utility execute the final call to the LLM for content generation? (Yes or no)
3. Was the backward-compatibility test designed to prove that the new Summarizer agent works correctly? (Yes or no)
4. Does the planner automatically learn to use the Summarizer agent just by updating the agent registry? (Yes or no)
5. Is the main business reason for introducing the Summarizer agent to increase the creativity of the final output? (Yes or no)
6. Does the Writer agent receive its factual input directly from the Summarizer agent in the new workflow? (Yes or no)

7. Did adding the Summarizer agent require significant changes to the core `engine.py` file? (Yes or no)
8. Is the `summary_objective` input used to provide the Summarizer agent with the full text to be condensed? (Yes or no)
9. Can proactive context management be described as the key strategic capability introduced in this chapter? (Yes or no)

## References

- Hao, S., Wang, Z., Li, J., Liu, Z., & Chen, C. (2024). AgentVerse: A Flexible Framework for Multi-Agent Society Simulation: *A Flexible Framework for Multi-Agent Society Simulation*. arXiv preprint arXiv:2405.08722. <https://arxiv.org/abs/2405.08722>
- Chen, L., Wu, Y., Zhang, Z., Xu, K., Wang, S., & Zhang, Y. (2024). AgentBoard: An LLM-Based Multi-Agent Platform for Proactive and Interactive Data Analysis: *An LLM-Based Multi-Agent Platform for Proactive and Interactive Data Analysis*. arXiv preprint arXiv:2407.03713. <https://arxiv.org/abs/2407.03713>
- Dasgupta, I., Hughes, E., Kuegel, A., Wang, F., Kay, J., & Collins, T. (2024). The Geometry of Context Management in Transformers *The Geometry of Context Management in Transformers*. arXiv preprint arXiv:2405.00205. <https://arxiv.org/abs/2405.00205>

## Further reading

Wang, L., Ma, W., Zhu, Y., Wang, Z., Liu, S., & Liu, J. (2024). A Survey on Large Language Model *A Survey on Large Language Model-based Autonomous Agents*. arXiv preprint arXiv:2308.11432. <https://arxiv.org/abs/2308.11432>

## Subscribe for a free eBook

New frameworks, evolving architectures, research drops, production breakdowns—*AI\_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and GenAI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at <https://packt.link/80z6Y> or scan the QR code below.



Table of contents collapsed